Marwadi University	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: Digital Design using Verilog (01CT1619)	**Aim:** Write the Verilog code for following Flipflops, (i) SR Flipflop and (ii) D Flipflop	
Experiment No: 11	**Date:**	**Enrollment No: 92301733025**

Aim: Write the Verilog code for following Flipfops, (i) SR Flipflop and (ii) D Flipfop

IDE: EDA Playground

Theory:

A flip flop in digital electronics is a circuit with two stable states that can be used to store binary data. The stored data can be changed by applying varying inputs. Flip-flops and latches are fundamental building blocks of digital electronics systems used in computers, communications, and many other types of systems. Both are used as data storage elements.

It is the basic storage element in sequential logic. But first, let's clarify the difference between a latch and flip-flops.

Flip Flop Types

There are basically 4 types of flip-flops in digital electronics:

1. SR Flip-Flop
2. JK Flip-Flop
3. D Flip-Flop
4. T Flip-Flop

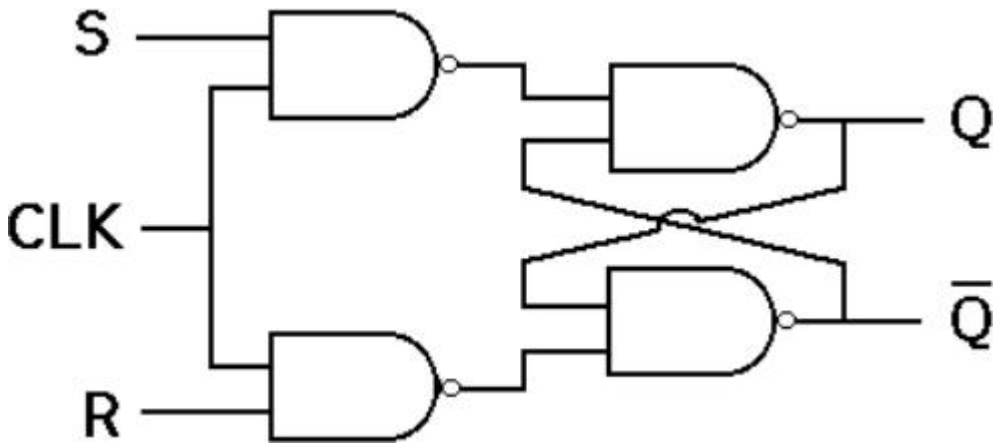
SR Flip Flop

This is the most common flip-flop among all. This simple flip-flop circuit has a set input (S) and a reset input (R). In this system, when you Set "S" as active, the output "Q" would be high, and "Q'" would be low. Once the outputs are established, the wiring of the circuit is maintained until "S" or "R" go high, or power is turned off.

As shown above, it is the simplest and easiest to understand. The two outputs, as shown above, are the inverse of each other. The **truth table of SR Flip-Flop** is highlighted below.

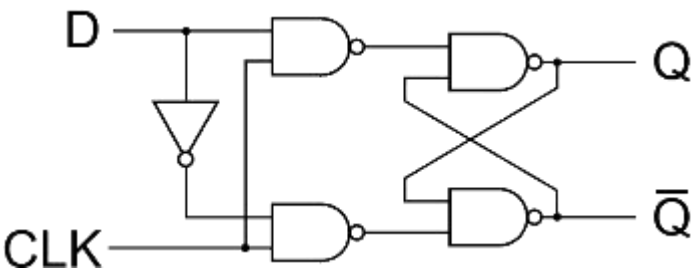
Marwadi University	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: Digital Design using Verilog (01CT1619)	**Aim:** Write the Verilog code for following Flipflops, (i) SR Flipflop and (ii) D Flipflop	
Experiment No: 11	**Date:**	**Enrollment No: 92301733025**

S	R	Q	Q′
0	0	0	1
0	1	0	1
1	0	1	0
1	1	∞	∞



D Flip-Flop

D flip-flop is a better alternative that is very popular with digital electronics. They are commonly used for counters and shift registers and input synchronization.



D Flip-Flop

Marwadi University	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: Digital Design using Verilog (01CT1619)	**Aim:** Write the Verilog code for following Flipflops, (i) SR Flipflop and (ii) D Flipflop	
Experiment No: 11	**Date:**	**Enrollment No: 92301733025**

In the D flip-flops, the output can only be changed at the clock edge, and if the input changes at other times, the output will be unaffected.

Clock	D	Q	Q′
↓ » 0	0	0	1
↑ » 1	0	0	1
↓ » 0	1	0	1
↑ » 1	1	1	0

Pre Lab Exercise:

a. What is FlipFlops?
A Flip-Flop is a digital electronic circuit used to store one bit of data (0 or 1). It is a bistable device, meaning it has two stable states. Flip-flops are used in sequential logic circuits where the output depends on both present input and previous output. Unlike latches, flip-flops are edge-triggered devices, which means their output changes only at the time of clock signal transition (positive or negative edge). Flip-flops are widely used in registers, counters, memory units, and digital systems.

b. what is difference between Latch and flipflops?

Latch	**Flip-Flop**
Latch is level-triggered device	Flip-flop is edge-triggered device
Output changes whenever input changes	Output changes only at clock edge
Does not require clock signal	Requires clock signal
Faster but less stable	More stable and reliable
Used in asynchronous circuits	Used in synchronous circuits
Example: SR latch, D latch	Example: SR flip-flop, JK flip-flop, D flip-flop

c. How to convert SR Flipflop into JK Flipflop?

An SR flip-flop can be converted into a JK flip-flop by using feedback connections from the outputs. In JK flip-flop, the invalid condition of SR flip-flop (when S = 1 and R = 1) is removed by using feedback.
Connections:
Connect input J to S input through feedback from $\overline{Q}$ (Q bar).
Connect input K to R input through feedback from Q.
Thus,
$S = J \cdot \overline{Q}$
$R = K \cdot Q$
When J = K = 1, the output toggles, which removes the invalid condition present in SR flip-flop. Therefore, SR flip-flop can be converted into JK flip-flop using feedback connections.

Verilog Code for SR flipflop:

```
// SR Flip-Flop
module sr_flipflop(
   input S,
   input R,
   input clk,
   output reg Q,
   output reg Qbar
);

always @(posedge clk)
begin
   case({S,R})

      2'b00: begin
               Q = Q;       // Hold
               Qbar = Qbar;
            end

      2'b01: begin
               Q = 0;       // Reset
               Qbar = 1;
            end

      2'b10: begin
               Q = 1;       // Set
```

```
            Qbar = 0;
        end

    2'b11: begin
            Q = 1'bx;     // Invalid
            Qbar = 1'bx;
        end

  endcase
end

endmodule
```

Test-bench for SR flipflop:

```
module tb_sr_flipflop;

reg S, R, clk;
wire Q, Qbar;

sr_flipflop uut(S, R, clk, Q, Qbar);

initial
begin

clk = 0;
forever #5 clk = ~clk;

end

initial
begin

$display("S R | Q Qbar");

S=0; R=0; #10;
$display("%b %b | %b %b", S, R, Q, Qbar);

S=1; R=0; #10;
$display("%b %b | %b %b", S, R, Q, Qbar);

S=0; R=1; #10;
```

```
$display("%b %b | %b %b", S, R, Q, Qbar);

S=1; R=1; #10;
$display("%b %b | %b %b", S, R, Q, Qbar);

$finish;

end

endmodule
```

Results for SR flipflop:

```
S R | Q Qbar
0 0 | x x
1 0 | 1 0
0 1 | 0 1
1 1 | x x
```

Verilog Code for D flipflop:

```
// D Flip-Flop
module d_flipflop(
   input D,
   input clk,
   output reg Q
);

always @(posedge clk)
begin
   Q = D;
end

endmodule
```

Test-bench for D flipflop:

```
module tb_d_flipflop;

reg D, clk;
wire Q;
```

```
d_flipflop uut(D, clk, Q);

initial
begin

clk = 0;
forever #5 clk = ~clk;

end

initial
begin

$display("D | Q");

D=0; #10;
$display("%b | %b", D, Q);

D=1; #10;
$display("%b | %b", D, Q);

D=0; #10;
$display("%b | %b", D, Q);

D=1; #10;
$display("%b | %b", D, Q);

$finish;

end

endmodule
```

Results for D flipflop:

```
D | Q
0 | 0
1 | 1
0 | 0
1 | 1
```

Marwadi University	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: Digital Design using Verilog (01CT1619)	**Aim:** Write the Verilog code for following Flipflops, (i) SR Flipflop and (ii) D Flipflop	
Experiment No: 11	**Date:**	**Enrollment No: 92301733025**

Observation and Result Analysis:

Flip-flops are important memory elements used in digital electronics to store one bit of information. They operate using clock signals, which makes them more stable and reliable than latches. The difference between latches and flip-flops lies mainly in their triggering method, where latches are level-triggered and flip-flops are edge-triggered. By using feedback connections, an SR flip-flop can be converted into a JK flip-flop, which removes the invalid condition and provides better performance. Hence, flip-flops are widely used in registers, counters, and other sequential logic circuits.

Post Lab Exercise:

a. Write verilog code, test bench and output for JK Flip flops

Verilog Code:

```
// JK Flip-Flop (positive edge triggered)

module jk_flipflop(
    input J,
    input K,
    input clk,
    output reg Q,
    output reg Qbar
);

always @(posedge clk)
begin

    case({J,K})

        2'b00: begin
                Q = Q;        // Hold
                Qbar = Qbar;
            end

        2'b01: begin
                Q = 0;        // Reset
                Qbar = 1;
            end

        2'b10: begin
                Q = 1;        // Set
                Qbar = 0;
```

```
            end

        2'b11: begin
                Q = ~Q;       // Toggle
                Qbar = ~Qbar;
            end

    endcase

end

endmodule
```

Testbench:

```
module tb_jk_flipflop;

reg J, K, clk;
wire Q, Qbar;

jk_flipflop uut(J, K, clk, Q, Qbar);

initial
begin

clk = 0;
forever #5 clk = ~clk;

end


initial
begin

$display("J K | Q Qbar");

// Hold
J=0; K=0; #10;
$display("%b %b | %b %b", J, K, Q, Qbar);

// Reset
J=0; K=1; #10;
```

```
$display("%b %b | %b %b", J, K, Q, Qbar);

// Set
J=1; K=0; #10;
$display("%b %b | %b %b", J, K, Q, Qbar);

// Toggle
J=1; K=1; #10;
$display("%b %b | %b %b", J, K, Q, Qbar);

// Toggle again
J=1; K=1; #10;
$display("%b %b | %b %b", J, K, Q, Qbar);

$finish;

end

endmodule
```

Results:

```
J K | Q Qbar
0 0 | x x
0 1 | 0 1
1 0 | 1 0
1 1 | 0 1
1 1 | 1 0
```